

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 5+6+9

Date : 08/11/2023
Matière : Programmation OO Java
Durée : 1h
Documents : Non autorisés
Nombre des pages : 2

Devoir Surveillé

Exercice 1

Qu'affiche le programme suivant ?

```
public class Exercice1{
    static{
        System.out.println("Message");
    }
    public static void main(String[] args) {
        int a=10;
        int b=011;
        int c=-3;
        System.out.println("b0 = "+b);
        System.out.println("Opération sur c = "+~c);
        var result = switch (a > b ? "grand" : "petit") {
            case "grand" -> ++a-++c+b++;
            case "petit" -> {
                int val = ++b*a--;
                yield (val+c);
            }
            default -> ++b;
        };
        System.out.println("Résultat = "+result);
        System.out.println("b1 = "+b);
        var op = (b != 10) && (a/b < 1) ? 1 : 2;
        switch (op) {
            case 0:
                System.out.println("Test 0");
                break;
            case 1:
                System.out.println("Test 1");
            case 2:
                System.out.println("Test 2");
            default:
                System.out.println("Test");
                break;
        }
    }
}
```

Message
b0 = 9
Opération sur c = 2
Résultat = 22
b1 = 10
Test 2
Test

Exercice 2

Écrivez un programme Java qui prend trois nombres réels (x, y et z) en ligne de commande et détermine le maximum et le minimum de ces trois nombres **sans utiliser** les méthodes max et min de la classe Math.

```
public class TestMaxMin {
    public static void main(String[] args) {
        if (args.length != 3) {
            System.out.println("Veuillez entrer trois nombres réels (x, y, z) en
ligne de commande :");
            return;
        }
        double x = Double.parseDouble(args[0]);
        double y = Double.parseDouble(args[1]);
        double z = Double.parseDouble(args[2]);

        double max = x;
        if (y > max) {
            max = y;
        }
        if (z > max) {
            max = z;
        }

        double min = x;
        if (y < min) {
            min = y;
        }
        if (z < min) {
            min = z;
        }

        System.out.println("Le maximum de " + x + ", " + y + " et " + z + " est " + max);
        System.out.println("Le minimum de " + x + ", " + y + " et " + z + " est " + min);
    }
}
```

Exercice 3

1. Écrivez une classe **Point** qui représente un point dans un espace bidimensionnel. Cette classe doit comporter les attributs et les méthodes suivants :

- Attributs :
 - double x : représentant l'abscisse du point.
 - double y : représentant l'ordonnée du point.
- Un constructeur prenant deux paramètres pour initialiser les coordonnées du point.
- Méthodes :
 - double getX() : renvoyant la coordonnée x du point.
 - double getY() : renvoyant la coordonnée y du point.
 - double distanceTo(Point otherPoint) : calculant la distance euclidienne entre le point courant et un autre point donné en tant que paramètre.
 - void translate(double dx, double dy) : effectuant une translation du point en déplaçant ses coordonnées de dx sur les abscisses et de dy sur les ordonnées.
 - void homothetie(double scaleFactor) : réalisant une homothétie du point par rapport à l'origine en multipliant ses coordonnées par un facteur d'échelle spécifié.

- static Point barycentre (Point[] points) : calculant et renvoyant les coordonnées du barycentre des points spécifiés dans le tableau. Le barycentre est calculé comme étant la moyenne des coordonnées de ces points.
2. Écrivez une classe **Segment** qui modélise un segment de ligne entre deux points. Cette classe doit comporter les attributs et les méthodes suivants :
- Attributs :
 - Point point1 : représentant le premier point d'extrémité du segment.
 - Point point2 : représentant le deuxième point d'extrémité du segment.
 - Un constructeur prenant deux références de type Point en tant que points d'extrémité du segment.
 - Méthodes :
 - Point getPoint1() : renvoyant le premier point d'extrémité du segment.
 - Point getPoint2() : renvoyant le deuxième point d'extrémité du segment.
 - double length() : calculant et renvoyant la longueur du segment, c'est-à-dire la distance entre les deux points d'extrémité.

Remarques :

- Respectez le **principe d'encapsulation** en déclarant les attributs et les méthodes.
- La distance euclidienne entre deux points (x1, y1) et (x2, y2) est égale à :

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

- La classe **Java.lang.Math** contient deux méthodes statiques permettant de calculer la racine carrée et la puissance d'un certain nombre.
Voici la déclaration de ces méthodes :

```
class Math {  
    public static double sqrt(double nb) :      // retourne la racine carrée de nb  
    public static double pow(double nb, double p) : // retourne nbp  
    ...  
}
```

```
public class Point {  
    private double x;  
    private double y;  
  
    public Point(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public double getX() {  
        return this.x;  
    }  
  
    public double getY() {  
        return this.y;  
    }  
  
    public double distanceTo(Point otherPoint) {  
        double dx = this.x - otherPoint.x;  
        double dy = this.y - otherPoint.y;  
        return Math.sqrt(dx * dx + dy * dy);  
    }  
}
```

```
    public void translate(double dx, double dy) {
        this.x += dx;
        this.y += dy;
    }

    public void homothetie(double scaleFactor) {
        this.x *= scaleFactor;
        this.y *= scaleFactor;
    }

    public static Point barycentre(Point[] points) {
        double sumX = 0;
        double sumY = 0;
        int n = points.length;
        for (Point p : points) {
            sumX += p.x;
            sumY += p.y;
        }
        return new Point(sumX / n, sumY / n);
    }
}
```

```
public class Segment {
    private Point point1;
    private Point point2;

    public Segment(Point point1, Point point2) {
        this.point1 = point1;
        this.point2 = point2;
    }

    public Point getPoint1() {
        return this.point1;
    }

    public Point getPoint2() {
        return this.point2;
    }

    public double length() {
        return this.point1.distanceTo(this.point2);
    }
}
```